

Chapter - 06

6.5 Exercises

6.1 Here, given,

Frequent closed itemsets = C

And support count of each of them = $\text{support}(X)$

In our algorithm we have to test a given itemset X is frequent or not.

So here X will be frequent, if X is a subset of a closed itemset, such as

$$X \subseteq Y \quad (Y \text{ is one of the } C)$$

and, $\text{support}(X) = \text{support}(Y)$

So the algorithm can be like

```
def check_itemset(x):
```

```
    for (each i in itemsets C):
```

```
        if (x ⊆ i and support(x) = support(i)):
```

```
            return support(x)
```

```
    return null
```

6.2 (a) No, just using generators, we can't determine support of all itemsets.

For example, if $\{A, B\}$ generator has support count 5, we can't know the support count of $\{A\}$, as it can be greater.

For this we need positive borders.

Positive borders can be itemsets such that

1. It is frequent
2. It is not a generator
3. Its subset is " "

The algorithm can be defined as

if (x in frequent or positive border):
 return support(x)
else:

for $\{x' \in x \text{ and } x' \in \text{positive border}\}$:

 if $\nexists x'' = x' - \{a\}$ where $x'' \in \text{frequent}$
 and $\text{support}(x'') = \text{support}(x')$
 $\& x = x' - \{a\}$

if (~~x is frequent~~ or positive border):
 return support(x)
 else
 return null

(b) Generator	Closed Itemset
1. An itemset is generator if no proper subset has the same support. 2. It has smallest itemset that produces a particular support.	1. An itemset X is closed if no proper superset has the same support. 2. It is the largest itemset that has a particular support.

(6.3) (a) Let's assume an itemset is frequent,

$$X = \{A, B, C\}$$

If $\{A, B, C\}$ appear in n transactions,
 then $\{A, B\}$ must appear in those as well

$\therefore$ So $\{A, B\}$ is also frequent.

(b) Let's assume

$$s = \{A, B, C\}$$

and $s' = \{A, B\}$

So now every transaction containing

$$\{A, B, C\}$$

must also has $\{A, B\}$

But there might be some transactions where C is missing. So if we

count then,

$$\text{sup_count}(\{A, B\}) \geq \text{sup_count}(\{A, B, C\})$$

$$\text{or, support}(\{A, B\}) \geq \text{support}(\{A, B, C\})$$

(a) Here, frequent itemset = I

and subset $s \subseteq I$

We have to prove,

$$c(s' \Rightarrow (I - s')) \leq c(s \Rightarrow (I - s))$$

Here, $s' \subseteq s$

Here, L.H.S. = confidence($s' \Rightarrow (1-s')$)

$$= \frac{\text{support}(s' \cup (1-s'))}{\text{support}(s')}$$

$$= \frac{\text{support}(I)}{\text{support}(s')}$$

$$= \frac{\text{support}(I)}{\text{support}(s)}$$

($\because$ because I contains everything)

R.H.S. = confidence($s \Rightarrow (1-s)$)

$$= \frac{\text{support}(s \cup (1-s))}{\text{support}(s)}$$

$$= \frac{\text{support}(I)}{\text{support}(s)}$$

We know, given is,

$$s' \subseteq s$$

so, $\text{support}(s') \geq \text{support}(s)$

or $\frac{\text{support}(I)}{\text{support}(s')} \leq \frac{\text{support}(I)}{\text{support}(s)}$

or, L.H.S. $\leq$ R.H.S

[Proved]

(d) proof by contradiction:

Let's say, x is frequent in the D

$$\text{support}_D(x) \geq \text{minsup}$$

But x is not frequent in any partition,

$$\text{support}_{D_i}(x) < \text{minsup}, \quad 1 \leq i \leq n$$

So, overall support will be,

$$\text{support}_D(x) = \sum_{i=1}^n \frac{N_i}{N} \text{support}_{D_i}(x)$$

Here,

$$\sum_{i=1}^n \frac{N_i}{N} = 1$$

As

$$\text{support}_{D_i}(x) < \text{minsup}$$

there average will also be less,

$$\text{support}_D(x) < \text{minsup}$$

But this is false as we said in the beginning.

So, if x is frequent in D , then it's frequent in at least one partition.

(6.4) Here c is generated from $(k-2)$ frequent subsets. As those two were frequent, as per (downward) rule, we do not need to check them again for consideration. So we only need to check $(k-2)$ items.

For example if

$$l_1 = \{A, B, c\}$$

$$l_2 = \{A, B, D\}$$

and they combine into,

$$c = \{A, B, c, D\}$$

then we don't need to check l_1 and l_2

(6.5) From previous examples, we know that,

$$\text{If } x \subseteq y$$

$$\text{then } \text{support}(x) \geq \text{support}(y)$$

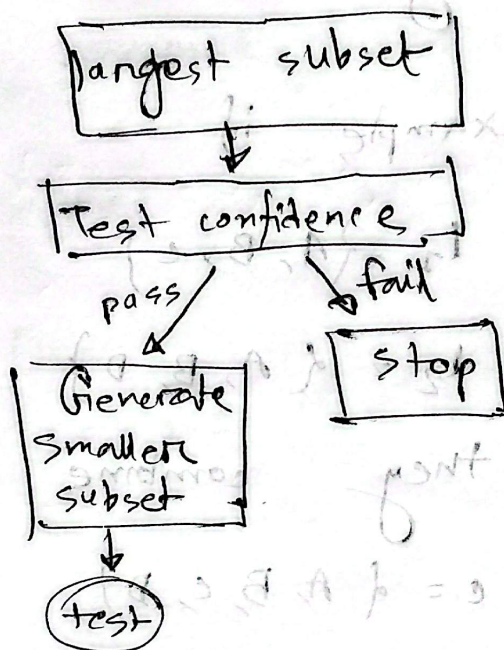
$$\text{and } \text{confidence}(x \Rightarrow (b-x)) \leq \text{confidence}(y \Rightarrow (1-y))$$

(2.0)

So our idea is, if $ABC \Rightarrow D$ fails then

and confidence $(ABC \Rightarrow D)$ fails then

its subset AB, AC, BC will also fail



(6.6)

(a)

Here given,

TID

items brought

T100	{M, O, N, K, E, Y}
T200	{D, O, N, K, E, Y}
T300	{M, A, K, E}
T400	{M, U, C, K, Y}
T500	{C, O, O, K, I, E}

There are 5 transaction, so for being frequent appearance need $60\% \times 5 = 3$

For apriori,

C_1

Itemset	Sup. Count
E	4
K	5
M	3
O	3
Y	3
N	2
D	1
A	1
U	1
C	2
I	2

Itemset	Sup
E	4
K	5
M	3
O	3
Y	3

Now, generating C_2

Itemset	sup. count
EK	4
EM	2
EO	3
EY	2
KM	3
KO	3
KY	3
MO	3
MY	1
OY	2

Itemset	Sup-count
EK	4
EO	3
KM	3
KO	3
KY	3

Generating	C_3
Itemset	sup-count
EKO	3

Generating C_4 isn't possible,
 so the frequent itemsets are

$\{E\}, \{K\}, \{M\}, \{O\}, \{Y\},$
 $\{E, K\}, \{E, O\}, \{K, M\}, \{K, O\}, \{K, Y\},$
 $\{E, K, O\}$

For FP-Growth,

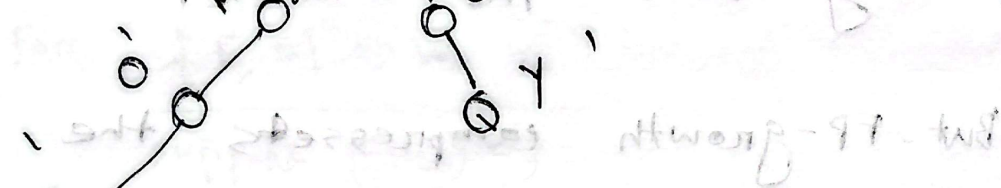
The L_2 will be

Itemset	sup-count
E	4
K	5
M	3
O	3
Y	3

Arranging in descend sup order

$K > E > M > O > Y$

It is important to note that the above information is for informational purposes only and should not be used for any other purpose.


$$(x, \text{mosti}(x))_{\text{epud}} \leftarrow (\text{mosti}(x))_{\text{epud}}$$
$$\{E\}, \{K\}, \{M\}, \{O\}, \{Y\}$$

$$\{K, Y\}, \{E, K, O\}, \{K, O\},$$

$$\{E, O\}, \{K, M\}, \{E, K\}$$

6.8 (a) the updated list of transactions,

tid	TID	items bought
1	T100	{Crab, Milk, Cheese, Bread}
2	T200	{Cheese, Milk, Apple, Pie, Bread}
1	T300	{Apple, Milk, Bread, Pie}
3	T400	{Bread, Milk, Cheese}

Using apriori,

C₁ TID items

Bread 4

Milk 4

Cheese 3

Apple 2

Pie 2

Crab 1

T₁
TID Items

Bread 4

Milk 4

Cheese 3

there minimum support count $4 \times 60\% \approx 3$

Then, C₂

TID items

Bread, Milk 4

Bread, cheese 3

Milk, cheese 3

And for frequent 3 itemsets,

TID items

Bread, Milk, Cheese 3

As per the question,

$\text{item}_1 \wedge \text{item}_2 \Rightarrow \text{item}_3$

we need 3 itemsets, Here,

Bread $\wedge$ cheese $\Rightarrow$ Milk

$$\text{support, } s = \frac{3}{4} = 75\%$$

$$\text{confidence, } c = \frac{3}{3} = 100\%$$

Cheese $\wedge$ Milk $\Rightarrow$ Bread

$$\text{confidence, } c = \frac{3}{3} = 100\%$$

Bread $\wedge$ Milk $\Rightarrow$ Cheese

$$\text{confidence, } c = \frac{3}{4} = 75\%$$

— This is not strong 75% < 80%

So, we can take cheese $\Rightarrow$ Bread $\wedge$ Milk

(b) For brand-item category,

custid	items_bought
1	{king's_crab, sunset-milk, Dairyland-cheese, Best-bread, Westcoast-Apple, Dairyland-Milk, Wonder-Bread, Tasty-Pie}
2	{Best-cheese, Dairyland-Milk, GoldenFarm-Apple, Tasty-Pie, Wonder-Bread}
3	{Wonder-Bread, Sunset-Milk, Dairyland-Cheese}

so the minimum support count = $3 \times 60\% \approx 2$

Here, C_1 cust-id item-count Li

cust-id	item-count	cust-id	sup-count
Wonder-Bread	3	Wonder-Bread	3
Dairyland-Milk	2	Dairyland Milk	2
Tasty-Pie	2	Tasty Pie	2
Sunset-Milk	2	Sunset Milk	2
Dairyland-cheese	2	Dairyland Cheese	2
king's Crab	1		
Best Bread	1		
West Coast Apple	1		
Best Cheese	1		
Golden Farm Apple	1		

Generating C_2

<u>Cust-id</u>	<u>sup_count</u>
WonderBread, DairylandMilk	2
WonderBread, TastyPie	2
WonderBread, SunsetMilk	2
WonderBread, DairylandCheese	2
DairylandMilk, TastyPie	2
DairylandMilk, SunsetMilk	1
DairylandMilk, DairylandCheese	1
TastyPie, SunsetMilk	1
TastyPie, DairylandCheese	1
SunsetMilk, DairylandCheese	2

And after pruning L_3

<u>Cust-id</u>	<u>sup_count</u>
WonderBread, DairylandMilk, TastyPie	2
WonderBread, SunsetMilk, DairylandCheese	2

(6.9) Here, if we consider M shops,

then,

global support count

$$\text{Count}_D(X) = \text{count}_1(X) + \dots + \text{count}_M(X)$$

So we can mine locally, and the n each site will send locally frequent itemsets and their support counts over the network.

And afterwards we can derive strong association rules from the global frequent itemsets.

(6.10) Let DB be the original transactional database. Let ΔDB be the newly added transactions. So the new database

$$DB' = DB + \Delta DB$$

Here instead of mining from scratch, we can do the following:

① Store the frequent itemsets and their support counts from the original dataset. It is also useful to set negative border.

② Construct a candidate set from the old frequent itemsets and the relevant negative-border itemsets.

③ Scan only the newly added transactions and count their occurrences.

④ Update the support count using,
$$\text{count}_{DB'}(X) = \text{count}_{DB}(X) + \text{count}_{ADB}(X)$$

⑤ Compute the new support using,
$$\text{support}_{DB'}(X) = \frac{\text{count}_{DB'}(X)}{|DB| + |ADB|}$$

⑥ If any candidate satisfies the minimum-support threshold, then that becomes frequent.

⑦ Finally generate association rules.

6.11

Let's say for example,
a transaction,

$T_1 = \{ \text{cake}, \text{cake}, \text{Milk} \}$

which is treated as $\{ \text{cake}, \text{Milk} \}$

But we want to preserve $\{ \text{cake}: 2, \text{Milk}: 1 \}$

So we can use quantity-aware mining. The idea is to treat quantity as a part of the item.

So above data will look like,

$\{ \text{cake}_1, \text{cake}_2, \text{Milk} \}$

Here, $\text{support}(\text{cake}_1), \text{support}(\text{cake}_2)$ will be used instead of single cake.

Likewise, for FP-Growth we can store quantity alongside, instead of creating duplicate nodes for efficiency.

(6.13)

Let's consider,

1000 total transition

Sunglass count = 900

Umbrella count = 100

Both sunglass & umbrella = 80

So if we use the rule

Umbrella $\rightarrow$ Sunglass

$$\text{confidence} = \frac{80}{100} = 80\%$$

If the minimum confidence is 70%, it is strong

But, if we look at probability,

$$P(\text{sunglass}) = \frac{900}{1000} = 90\%$$

$$\text{But } P(\text{sunglass} | \text{Umbrella}) = \frac{80}{100} = 80\%$$

Here, $P(\text{sunglass} | \text{umbrella}) < P(\text{sunglasses})$

So they are negatively correlated

(C.14) (a) Here the given table

	Hot dog	not dog	
hamburgers	2000	500	2500
hamburgers	1000	1500	2500
	3000	2000	5000

Given rule,

Hot dogs $\Rightarrow$ hamburger

Here, support $= \frac{2000}{5000} = 40\%$

confidence $= \frac{2000}{3000} = 66.7\%$

They meet the requirements,

so the rule is strong

(b) To learn the correlation between hotdog and hamburger

hotdog $\Rightarrow$ hamburger

$$\begin{aligned} \text{Corr}(\text{hotdog}, \text{hamburger}) &= \frac{P(\text{hotdog}, \text{hamburger})}{P(\text{hotdog}) P(\text{hamburger})} \\ &= \frac{0.4}{0.5 \times 0.6} \\ &= 1.33 > 1 \end{aligned}$$

So they are not independent,
a positive correlation exists.

(c) Let's first calculate
all-confidence
max-confidence
kulczynski
cosine measures